

Praktikum 5:

JAVASCRIPT

Sub Materi:

1. Pengenalan JavaScript
2. Menambahkan JavaScript ke HTML
3. Variabel dan Tipe Data
4. Operator dan Ekspresi
5. Percabangan
6. Perulangan
7. Function (Fungsi)
8. DOM dan Event
9. Form dan Validasi Sederhana

A. TUJUAN PRAKTIKUM

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Mengintegrasikan JavaScript dengan HTML untuk menambahkan perilaku dan interaksi pada halaman web.
2. Menerapkan variabel, tipe data, dan operator dalam program JavaScript sederhana.
3. Menerapkan percabangan, perulangan, dan function untuk mengatur proses program.
4. Memanipulasi elemen HTML dan menangani event menggunakan DOM dan JavaScript.
5. Mengolah dan memvalidasi data form serta menampilkan hasilnya secara dinamis

B. TEORI DASAR

1. Pengenalan JavaScript

JavaScript adalah bahasa pemrograman yang digunakan untuk menambahkan perilaku dan interaksi pada halaman web. JavaScript memungkinkan sebuah halaman web merespons tindakan pengguna dan melakukan perubahan terhadap isi maupun tampilan halaman secara dinamis.

Pada awal pengembangan web, halaman umumnya digunakan untuk menampilkan informasi yang bersifat statis. Informasi yang ditampilkan tidak banyak berubah selama pengguna tidak berpindah halaman atau melakukan proses tertentu. JavaScript memungkinkan halaman web melakukan proses berdasarkan tindakan pengguna sehingga interaksi dapat dilakukan secara langsung pada halaman.

Sebagai contoh, ketika pengguna menekan sebuah tombol, JavaScript dapat digunakan untuk menampilkan pesan. Ketika pengguna memasukkan data, JavaScript dapat mengambil data tersebut, melakukan perhitungan, memeriksa kondisi, atau menampilkan hasil tertentu. Dengan kemampuan tersebut, JavaScript memungkinkan halaman web memberikan respons terhadap tindakan pengguna.

Dalam pengembangan web, JavaScript biasanya digunakan bersama HTML dan CSS. Ketiganya memiliki fungsi yang berbeda, tetapi saling melengkapi.

Tabel 5.1 Peran HTML, CSS, dan JavaScript

Teknologi	Peran	Contoh Penerapan
HTML	Membentuk struktur dan konten	Judul, paragraf, gambar, tabel, form
CSS	Mengatur tampilan	Warna, ukuran, layout, jarak
JavaScript	Mengatur perilaku dan interaksi	Tombol, validasi, perhitungan, perubahan konten

HTML menentukan elemen apa saja yang terdapat pada halaman. CSS menentukan bagaimana elemen tersebut ditampilkan. JavaScript menentukan bagaimana halaman memberikan respons terhadap tindakan pengguna.

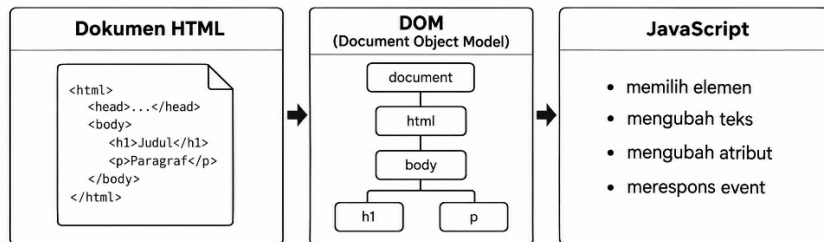
Penggunaan JavaScript tersebut memungkinkan sebuah halaman web memberikan pengalaman interaksi yang lebih dinamis. Proses yang dilakukan JavaScript dapat berlangsung berdasarkan tindakan pengguna maupun kondisi tertentu yang terjadi pada program. Dalam pengembangan web, JavaScript dapat digunakan untuk berbagai kebutuhan, antara lain:

- a. Membuat Halaman Web Menjadi Interaktif;
- b. Memberikan Respons Terhadap Tindakan Pengguna;
- c. Melakukan Perhitungan;
- d. Memproses Data;
- e. Mengubah Isi Dokumen Html;
- f. Mengubah Atribut Elemen Html;
- g. Mengubah Tampilan Halaman Secara Dinamis;
- h. Melakukan Validasi Data;
- i. Mengatur Alur Proses Berdasarkan Kondisi Tertentu; Dan
- j. Membuat Komponen Antarmuka Yang Memberikan Respons Terhadap Pengguna.

JavaScript tidak hanya digunakan untuk menjalankan perhitungan atau proses logika, tetapi juga dapat digunakan untuk berkomunikasi dengan elemen yang terdapat pada dokumen HTML.

Pada browser, JavaScript dapat berinteraksi dengan dokumen HTML melalui Document Object Model (DOM). DOM merepresentasikan dokumen sebagai objek yang dapat diakses dan dimanipulasi oleh program. Melalui DOM, JavaScript dapat memperoleh elemen HTML, membaca atau mengubah

isi elemen, mengubah atribut, menambahkan atau menghapus elemen, serta merespons event yang terjadi pada halaman.



Gambar 5.1 Document Object Model (DOM)

Dokumen HTML dibaca oleh browser dan direpresentasikan dalam bentuk DOM. JavaScript kemudian dapat mengakses representasi tersebut untuk melakukan manipulasi terhadap halaman.

2. Menambahkan JavaScript ke HTML

JavaScript dapat diintegrasikan ke dalam dokumen HTML dengan menggunakan elemen `<script>`. Elemen `<script>` berfungsi sebagai penghubung antara dokumen HTML dan kode JavaScript, baik ketika kode JavaScript ditulis langsung di dalam dokumen HTML maupun ketika kode tersebut disimpan dalam file eksternal.

Berdasarkan cara penempatannya, JavaScript dapat ditulis secara inline, internal, atau external. Pemilihan cara penempatan JavaScript perlu disesuaikan dengan kebutuhan program. Pada contoh yang sangat sederhana, JavaScript dapat ditulis langsung pada elemen HTML atau di dalam dokumen HTML. Untuk proyek yang lebih besar, JavaScript dapat dipisahkan ke dalam file tersendiri.

a. Inline JavaScript

Inline JavaScript adalah JavaScript yang ditulis langsung pada elemen HTML melalui atribut tertentu. Pendekatan ini memungkinkan suatu tindakan sederhana dilakukan langsung dari elemen HTML. Cara tersebut relatif mudah dipahami karena kode JavaScript ditempatkan langsung pada elemen yang akan menjalankan tindakan.

```
<elemen atribut_event="kode JavaScript">
    Konten elemen
</elemen>
```

Contohnya:

```
<button onclick="alert('Halo!')">
    Klik
</button>
```

Namun, penggunaan inline JavaScript membuat struktur HTML bercampur dengan kode JavaScript. Jika jumlah kode semakin banyak, kondisi tersebut dapat menyebabkan kode lebih sulit dibaca dan dipelihara. Oleh karena itu, inline JavaScript lebih sesuai digunakan untuk contoh yang sangat sederhana atau untuk memahami konsep awal interaksi antara HTML dan JavaScript.

b. Internal JavaScript

Internal JavaScript adalah JavaScript yang ditulis di dalam dokumen HTML menggunakan elemen `<script>`. JavaScript ditempatkan dalam dokumen HTML yang sama dengan struktur halaman. Cara ini dapat digunakan apabila kode JavaScript hanya diperlukan pada satu halaman dan jumlah kode masih relatif sedikit.

```
<script>
    // kode JavaScript
</script>
```

Contohnya:

```
<script>
    alert("Selamat datang!");
</script>
```

Pemisahan kode JavaScript dari atribut elemen HTML membuat struktur kode lebih terorganisasi dibandingkan penggunaan inline JavaScript. Namun, apabila kode yang sama diperlukan oleh beberapa halaman, penulisan internal dapat menyebabkan kode harus ditulis berulang kali.

c. External JavaScript

External JavaScript adalah JavaScript yang disimpan dalam file terpisah dengan ekstensi .js. File tersebut kemudian dihubungkan dengan dokumen HTML. Penggunaan file eksternal memiliki beberapa keuntungan, antara lain:

- 1) kode lebih terorganisasi;
- 2) kode JavaScript dapat digunakan pada beberapa halaman;
- 3) pemeliharaan kode menjadi lebih mudah;
- 4) struktur proyek menjadi lebih teratur.

```
<script src="nama-file.js"></script>
```

Atribut src digunakan untuk menentukan lokasi file JavaScript yang akan dihubungkan dengan dokumen HTML. Nama file dan lokasinya disesuaikan dengan struktur folder proyek. Dalam penerapannya, pengembang perlu memperhatikan lokasi pemuatan script dan waktu ketika elemen HTML tersedia. Hal ini berkaitan dengan cara JavaScript mengakses elemen pada dokumen.

Pendekatan external JavaScript memberikan pemisahan yang lebih jelas antara struktur HTML, tampilan CSS, dan logika JavaScript. Dalam pengembangan website yang lebih besar, pemisahan HTML, CSS, dan JavaScript menjadi penting agar pengelolaan kode lebih mudah dilakukan.

3. Variabel dan Tipe Data

Program membutuhkan tempat untuk menyimpan data yang akan digunakan selama proses berlangsung. Dalam JavaScript, data dapat disimpan menggunakan variabel. Variabel dapat digunakan untuk menyimpan berbagai

jenis nilai, seperti nama, angka, status, hasil perhitungan, dan data lainnya. Nilai yang disimpan dapat digunakan kembali dalam proses program. navigation, content, sidebar, card, footer ditempatkan pada halaman.

a. Variabel

Variabel dapat dipahami sebagai sebuah nama yang digunakan untuk merujuk pada suatu nilai. Nilai tersebut dapat digunakan kembali dalam proses program. JavaScript menyediakan tiga keyword untuk deklarasi variabel, yaitu:

1) Let

let digunakan untuk mendeklarasikan variabel yang nilainya dapat diberikan kembali selama program berjalan. Variabel dengan let sesuai digunakan ketika nilai suatu data dapat berubah akibat proses program.

Contoh:

```
let nilai = 80;  
nilai = 90;
```

2) Const

const digunakan untuk mendeklarasikan binding yang tidak dapat diberikan nilai baru setelah deklarasi. const sesuai digunakan ketika suatu variabel tidak perlu dilakukan reassignment. Contoh:

```
Const nama ="Andi";
```

3) Var

var merupakan bentuk deklarasi variabel yang telah tersedia sejak versi JavaScript sebelumnya. var memiliki aturan scope yang berbeda dari let dan const.

Dalam penulisan JavaScript modern, let dan const lebih umum digunakan karena keduanya memiliki aturan block scope. Artinya, variabel yang dideklarasikan dengan let atau const hanya dapat digunakan di dalam blok kode tempat variabel tersebut dideklarasikan. Aturan ini membantu

membatasi penggunaan variabel sehingga struktur program menjadi lebih teratur dan mengurangi kemungkinan terjadinya konflik atau perubahan nilai variabel yang tidak diinginkan.

b. Tipe Data

Tipe data menunjukkan jenis nilai yang digunakan dan disimpan oleh suatu program. Setiap nilai dalam JavaScript memiliki tipe tertentu yang menentukan bagaimana nilai tersebut dapat digunakan dalam program

JavaScript merupakan bahasa pemrograman yang memiliki berbagai tipe data. Secara umum, tipe data dalam JavaScript dapat dibedakan menjadi tipe data primitif (primitive data types) dan tipe data non-primitif, yang pada JavaScript terutama direpresentasikan oleh objek.

1) Tipe Data Primitif

Tipe data primitif merupakan tipe data dasar yang merepresentasikan satu nilai. Nilai primitif bersifat sederhana dan bukan merupakan objek. JavaScript memiliki tujuh tipe data primitif, yaitu:

Tabel 5.2 Jenis-Jenis Tipe Data Primitif

No.	Tipe Data	Pengertian	Contoh
1	String	Digunakan untuk menyimpan teks atau kumpulan karakter.	"informatika"
2	Number	Digunakan untuk menyimpan bilangan bulat maupun bilangan pecahan.	85, 90.5
3	Boolean	Digunakan untuk menyimpan nilai logika yang terdiri atas true atau false.	true
4	Null	Menunjukkan bahwa suatu variabel secara sengaja tidak memiliki nilai.	null

5	Undefined	Menunjukkan bahwa suatu variabel telah dideklarasikan tetapi belum memiliki nilai.	undefined
6	BigInt	Digunakan untuk menyimpan bilangan bulat dengan ukuran yang sangat besar.	12345678901 234567890n
7	Symbol	Digunakan untuk menghasilkan nilai yang bersifat unik, terutama sebagai identifier atau property key.	Symbol("id")

2) Tipe Data Non-Primitif (Objek)

Tipe data non-primitif digunakan untuk merepresentasikan data yang memiliki struktur lebih kompleks dan dapat terdiri atas kumpulan nilai atau properti. Dalam JavaScript, tipe data non-primitif pada dasarnya berupa Object. Object dapat digunakan untuk mengelompokkan beberapa data dalam satu struktur.

Tabel 5.3 Jenis-Jenis Tipe Data Non-Primitif

No.	Tipe Data	Pengertian	Contoh
1	Object	Digunakan untuk menyimpan data yang terdiri atas beberapa property dalam bentuk pasangan key dan value.	{nama: "Budi", nilai: 85}
2	Array	Digunakan untuk menyimpan sekumpulan nilai dalam urutan tertentu. Setiap nilai memiliki index yang dimulai dari 0.	[80, 85, 90]

4. Operator dan Ekspresi

Operator digunakan untuk melakukan operasi terhadap data. Dalam JavaScript terdapat berbagai jenis operator, seperti operator aritmatika, perbandingan, logika, dan assignment. Kombinasi antara nilai, variabel, dan

operator dapat membentuk sebuah **ekspresi**. Ekspresi menghasilkan suatu nilai yang dapat digunakan dalam proses program.

a. Operator Aritmatika

Operator aritmatika digunakan untuk melakukan operasi matematika. Operator aritmatika dapat digunakan dalam proses seperti menghitung total harga, luas, nilai rata-rata, dan berbagai perhitungan lainnya.

Tabel 5.4 Operator Aritmatika

Operator	Fungsi
+	Penjumlahan
-	Pengurangan
*	Perkalian
/	Pembagian
%	Sisa pembagian
**	Perpangkatan

b. Operator Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai. Hasil perbandingan berupa nilai 10oolean. Dalam penulisan JavaScript, pemahaman terhadap perbedaan == dan === penting karena keduanya memiliki aturan perbandingan yang berbeda.

Tabel 5.5 Operator Perbandingan

Operator	Fungsi
==	Sama setelah proses konversi tertentu
===	Sama nilai dan tipe
!=	Tidak sama
!==	Tidak sama nilai atau tipe
>	Lebih besar
<	Lebih kecil
>=	Lebih besar atau sama dengan
<=	Lebih kecil atau sama dengan

c. Operator Logika

Operator logika digunakan untuk menggabungkan beberapa kondisi. Operator logika banyak digunakan dalam percabangan dan validasi.

Tabel 5.6 Operator Logika

Operator	Nama	Fungsi
&&	AND	Menghasilkan true jika semua kondisi bernilai true.
	OR	Menghasilkan true jika salah satu atau semua kondisi bernilai true.
!	NOT	Membalik nilai logika.

d. Operator Assignment

Operator assignment digunakan untuk memberikan nilai atau memperbarui nilai suatu variable.

Tabel 5.7 Operator Assignment

Operator	Fungsi
=	Assignment
+=	Menambah nilai
-=	Mengurangi nilai
*=	Mengalikan nilai
/=	Membagi nilai

e. Increment dan Decrement

Increment digunakan untuk meningkatkan nilai sebesar satu, sedangkan decrement digunakan untuk mengurangi nilai sebesar satu. Operator ini banyak digunakan dalam struktur perulangan.

Tabel 5.8 Operator Increment dan Decrement

Operator	Nama	Fungsi
++	Increment	Menambah nilai variabel sebanyak 1.
--	Decrement	Mengurangi nilai variabel sebanyak 1.

5. Percabangan dan Perulangan

Program tidak selalu menjalankan semua instruksi dengan cara yang sama. Dalam kondisi tertentu, program perlu menentukan tindakan berdasarkan suatu kondisi. Pada kondisi lainnya, program perlu menjalankan suatu proses secara berulang.

a. Percabangan

Percabangan digunakan untuk menentukan proses yang dijalankan berdasarkan kondisi tertentu. Program dapat menjalankan blok kode yang berbeda sesuai dengan hasil pemeriksaan kondisi.

Tabel 5.9 Jenis-jenis Percabangan

Jenis	Pengertian	Struktur Dasar
if	Menjalankan blok kode apabila suatu kondisi bernilai true.	<pre>if (kondisi) { // kode yang dijalankan }</pre>
if...else	Menentukan salah satu dari dua proses berdasarkan kondisi.	<pre>if (kondisi) { // kode jika kondisi true } else { // kode jika kondisi false }</pre>
if.. else if.. else	Memeriksa beberapa kondisi secara berurutan dan menjalankan blok kode sesuai kondisi yang terpenuhi.	<pre>if (kondisi1) { // kode jika kondisi1 true } else if (kondisi2) { // kode jika kondisi2 true } else { // kode jika semua kondisi false }</pre>
switch	Memilih proses berdasarkan nilai suatu ekspresi. case menentukan nilai yang diperiksa, break menghentikan proses, dan default dijalankan jika tidak ada nilai yang sesuai.	<pre>switch (ekspresi) { case nilai1: // kode untuk nilai1 break; case nilai2: // kode untuk nilai2 break; default: // jika tidak ada yang sesuai }</pre>

b. Perulangan

Perulangan digunakan untuk menjalankan suatu blok kode secara berulang selama kondisi atau ketentuan tertentu terpenuhi. JavaScript menyediakan beberapa bentuk perulangan, di antaranya for dan while.

Tabel 5.10 Jenis-jenis Perulangan

Jenis	Pengertian	Struktur Dasar
for	Menjalankan blok kode secara berulang berdasarkan inisialisasi, kondisi, dan perubahan nilai.	<pre>for (inisialisasi; kondisi; perubahan) { // kode yang diulang }</pre>
while	Menjalankan blok kode selama kondisi bernilai true.	<pre>while (kondisi) { // kode yang diulang }</pre>

6. Function (Fungsi)

Function digunakan untuk mengelompokkan suatu proses agar dapat digunakan kembali. Sebuah function dapat menerima input, melakukan proses, dan menghasilkan output. Struktur dasar function dalam JavaScript dapat dituliskan sebagai berikut:

```
function namaFunction(parameter) {  
    // proses yang dijalankan  
    return nilai;  
}
```

Contohnya:

```
function luas(panjang, lebar) {  
    return panjang * lebar;  
}
```

7. DOM dan Event

Document Object Model (DOM) merupakan model yang merepresentasikan dokumen HTML sebagai struktur objek. DOM memungkinkan JavaScript mengakses dan memanipulasi elemen HTML sehingga halaman web dapat berubah dan merespons tindakan pengguna.

DOM menjadi penghubung antara JavaScript dan elemen HTML. Melalui DOM, JavaScript dapat memilih, membaca, mengubah, menambahkan, dan menghapus elemen HTML. JavaScript juga dapat mengubah atribut dan style elemen secara dinamis.

JavaScript perlu memilih elemen HTML sebelum melakukan manipulasi. Elemen dapat dipilih berdasarkan id, class, nama elemen, atau selector CSS.

Tabel 5.11 Metode Pemilihan Elemen HTML

Cara Memilih	Keterangan
id	Memilih elemen berdasarkan nilai id yang dimiliki elemen HTML.
class	Memilih elemen berdasarkan nilai class.
Nama elemen	Memilih elemen berdasarkan nama tag HTML, seperti p, div, atau button.
Selector CSS	Memilih elemen menggunakan pola selector seperti yang digunakan dalam CSS.

Event merupakan kejadian pada halaman yang dapat ditangani oleh JavaScript, baik yang berasal dari tindakan pengguna maupun kejadian tertentu pada browser. Event dapat ditangani menggunakan `addEventListener()`, yaitu mekanisme untuk menentukan fungsi yang dijalankan ketika event tertentu terjadi. Beberapa event dasar yang perlu dipahami adalah:

Tabel 5.12 Jenis-Jenis Event Dasar

Event	Penggunaan
click	Terjadi ketika pengguna mengklik suatu elemen.
submit	Terjadi ketika pengguna mengirimkan form.
change	Terjadi ketika nilai suatu elemen berubah.
input	Terjadi ketika nilai input berubah saat pengguna memasukkan data.

8. Form dan Validasi Sederhana

Form merupakan bagian penting dalam aplikasi web karena digunakan untuk menerima data dari pengguna. Contoh penggunaan form antara lain form pendaftaran, login, kontak, pencarian, dan pengisian data mahasiswa.

JavaScript dapat digunakan untuk membaca data dari form, memeriksa data, memberikan pesan, dan menampilkan hasil pemrosesan.

a. Mengambil Nilai Input

Ketika pengguna mengisi sebuah form, nilai yang dimasukkan dapat diakses oleh JavaScript. Kemampuan mengambil nilai input menjadi dasar bagi proses pengolahan form menggunakan JavaScript. Data tersebut dapat digunakan untuk berbagai proses, seperti:

- 1) Melakukan perhitungan;
- 2) memeriksa kelengkapan data;
- 3) menampilkan kembali data;
- 4) mengolah informasi sesuai kebutuhan aplikasi.

b. Validasi Form

Validasi form merupakan proses pemeriksaan data untuk memastikan bahwa data yang dimasukkan memenuhi aturan yang telah ditentukan. Beberapa contoh aturan validasi adalah:

- 1) nama harus diisi;
- 2) email harus diisi;
- 3) data harus memiliki format tertentu;
- 4) angka harus berada dalam rentang tertentu;
- 5) pilihan tertentu harus dipilih.

HTML telah menyediakan mekanisme validasi bawaan melalui atribut tertentu, seperti `required`, `min`, `max`, `minlength`, `maxlength`, dan tipe input tertentu. JavaScript dapat digunakan apabila diperlukan pemeriksaan atau perilaku tambahan. Validasi membantu memastikan bahwa data yang diberikan pengguna sesuai dengan ketentuan yang telah ditetapkan sebelum data diproses lebih lanjut.

c. Menampilkan Pesan Validasi

Apabila data tidak memenuhi aturan, program perlu memberikan informasi kepada pengguna. Pesan validasi dapat digunakan untuk menjelaskan:

- 1) bagian data yang belum diisi;
- 2) kesalahan format;
- 3) data yang tidak sesuai ketentuan;
- 4) tindakan yang perlu dilakukan pengguna.

Pesan yang jelas membantu pengguna memperbaiki kesalahan input. Dalam halaman web interaktif, pesan validasi dapat ditampilkan secara dinamis menggunakan JavaScript sehingga pengguna dapat memperoleh umpan balik tanpa harus berpindah ke halaman lain.

d. Manipulasi Hasil

Setelah data dinyatakan valid, JavaScript dapat digunakan untuk menampilkan hasil proses pada halaman. Sebagai contoh, pada form pendaftaran mahasiswa, JavaScript dapat mengambil nama, program studi, dan data lainnya kemudian menampilkannya kembali sebagai informasi konfirmasi.

C. ALAT DAN BAHAN

1. Komputer atau laptop.
2. Web browser, seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge.
3. Text editor: Visual Studio Code
4. Folder kerja praktikum di akun github.
5. File multimedia untuk latihan menampilkan multimedia.

D. PRAKTIKUM

1. LATIHAN

a. Latihan 1: Variabel dan Operator

1) Kode Program

Latihan1.html	
1	<!DOCTYPE html>
2	<html>

3	<head>
4	<title>Variabel dan Operator</title>
5	</head>
6	<body>
7	
8	Perhitungan Nilai
9	<p id="hasil"></p>
10	
11	<script>
12	let nilaiTugas = 80;
13	let nilaiUjian = 90;
14	
15	let rataRata = (nilaiTugas + nilaiUjian) / 2;
16	
17	document.getElementById("hasil").textContent =
18	"Nilai rata-rata: " + rataRata;
19	</script>
20	
21	</body>
22	</html>

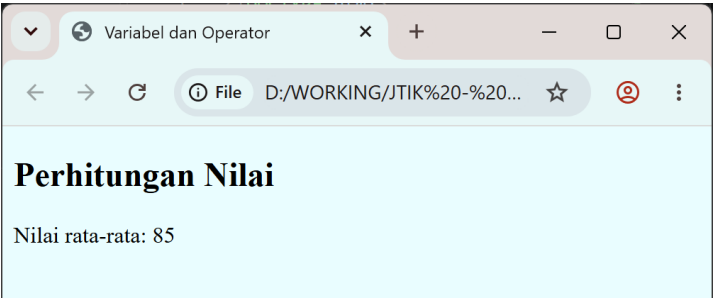
2) Analisis Kode Program

Tabel 5.13 Analisis Program Latihan 1 – Variabel dan Operator

Kode Program	Analisis
let nilaiTugas = 80;	Mendeklarasikan variabel nilaiTugas menggunakan let dan memberikan nilai awal 80. Variabel ini digunakan untuk menyimpan nilai tugas mahasiswa.
let nilaiUjian = 90;	Mendeklarasikan variabel nilaiUjian dengan nilai awal 90. Nilai tersebut akan digunakan dalam proses perhitungan.
let rataRata = (nilaiTugas + nilaiUjian) / 2;	Menghitung nilai rata-rata dengan menjumlahkan nilaiTugas dan nilaiUjian, kemudian membaginya dengan 2. Hasil perhitungan disimpan dalam variabel rataRata.
document.getElementById("hasil")	Memilih elemen HTML yang memiliki id="hasil" menggunakan DOM. Elemen tersebut menjadi tempat untuk menampilkan hasil perhitungan.

<pre>.textContent = "Nilai rata-rata: " + rataRata;</pre>	Mengubah isi teks elemen hasil. Operator + digunakan untuk menggabungkan teks dengan nilai yang tersimpan dalam rataRata.
---	--

3) Hasil



Gambar 5.2 Hasil Eksekusi Latihan 1 – Variabel dan Operator

b. Latihan 2: Percabangan

1) Kode Program

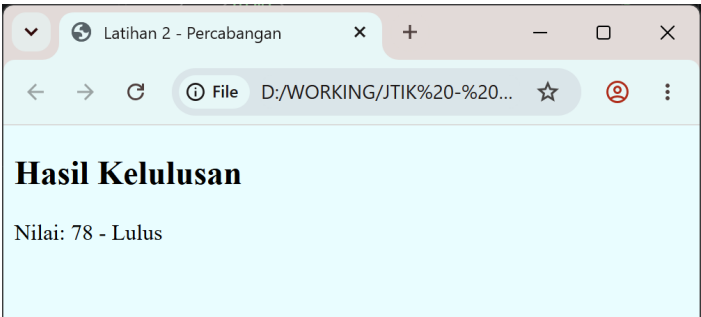
Latihan2.html	
1	<!DOCTYPE html>
2	<html>
3	<head>
4	<title>Percabangan</title>
5	</head>
6	<body>
7	
8	Hasil Kelulusan
9	<p id="hasil"></p>
10	
11	<script>
12	let nilai = 78;
13	let keterangan;
14	
15	if (nilai >= 75) {
16	keterangan = "Lulus";
17	} else {
18	keterangan = "Tidak Lulus";
19	}
20	
21	document.getElementById("hasil").textContent =
22	"Nilai: " + nilai + " - " + keterangan;
23	</script>
24	
25	</body>
26	</html>

2) Analisis Kode Program

Tabel 5.14 Analisis Program Latihan 2 – Percabangan

Kode Program	Analisis
<code>let nilai = 78;</code>	Mendeklarasikan variabel nilai untuk menyimpan nilai mahasiswa, yaitu 78.
<code>let keterangan;</code>	Mendeklarasikan variabel keterangan yang akan digunakan untuk menyimpan hasil pemeriksaan kelulusan.
<code>if (nilai >= 75) {</code>	Memeriksa apakah nilai lebih besar atau sama dengan 75. Operator <code>>=</code> digunakan untuk melakukan perbandingan.
<code>keterangan = "Lulus";</code>	Jika kondisi bernilai true, variabel keterangan diberi nilai "Lulus".
<code>} else {</code>	Menyediakan alternatif proses apabila kondisi pada if bernilai false.
<code>keterangan = "Tidak Lulus";</code>	Jika nilai kurang dari 75, variabel keterangan diberi nilai "Tidak Lulus".
<code>document.getElementById("hasil")</code>	Memilih elemen dengan id="hasil" sebagai tempat menampilkan hasil.
<code>.textContent = "Nilai: " + nilai + " - " + keterangan;</code>	Menampilkan nilai dan keterangan kelulusan pada halaman. Beberapa nilai String digabungkan menggunakan operator <code>+</code> .

3) Hasil



Gambar 5.3 Hasil Eksekusi Latihan 2 - Percabangan

c. Latihan 3: Perulangan dan Function

1) Kode Program

Latihan3.html	
1	<!DOCTYPE html>
2	<html>
3	<head>
4	<title>Latihan 3 - Perulangan dan Function</title>
5	</head>
6	<body>
7	
8	Daftar Nilai
9	<p id="hasil"></p>
10	
11	<script>
12	function tampilkanNilai(nilai) {
13	return "Nilai mahasiswa: " + nilai;
14	}
15	
16	let dataNilai = [80, 85, 90];
17	let hasil = "";
18	
19	for (let i = 0; i < dataNilai.length; i++) {
20	hasil += tampilkanNilai(dataNilai[i]) + " ";
21	}
22	
23	document.getElementById("hasil").innerHTML = hasil;
24	</script>
25	
26	</body>
27	</html>

2) Analisis Kode Program

Tabel 5.15 Analisis Program Latihan 3 – Perulangan dan Function

Kode Program	Analisis
function tampilkanNilai(nila i) {	Mendeklarasikan function bernama tampilkanNilai. Function memiliki parameter nilai untuk menerima data.
return "Nilai mahasiswa: " + nilai;	Mengembalikan String yang berisi keterangan dan nilai yang diterima oleh parameter.
let dataNilai = [80, 85, 90];	Membuat Array bernama dataNilai yang berisi tiga nilai. Setiap data memiliki index yang dimulai dari 0.

<pre>let hasil = "";</pre>	Membuat variabel hasil sebagai tempat untuk mengumpulkan teks yang akan ditampilkan.
<pre>for (let i = 0; i < dataNilai.length; i++) {</pre>	Membuat perulangan for. Variabel i dimulai dari 0 dan bertambah satu setiap perulangan. Perulangan berlangsung selama i lebih kecil dari jumlah data dalam Array.
<pre>dataNilai[i]</pre>	Mengambil nilai Array berdasarkan index yang sedang diproses. Misalnya, saat i bernilai 0, program mengambil nilai 80.
<pre>tampilkanNilai(dataNilai[i])</pre>	Memanggil function tampilkanNilai() dan memberikan nilai Array sebagai argument.
<pre>hasil += ...</pre>	Menambahkan hasil dari function ke variabel hasil. Operator += digunakan agar hasil sebelumnya tetap tersimpan.
<pre>"
"</pre>	Menambahkan perpindahan baris HTML agar setiap nilai ditampilkan pada baris yang berbeda.
<pre>document.getElementById("hasil")</pre>	Memilih elemen HTML dengan id="hasil".
<pre>.innerHTML = hasil;</pre>	Memasukkan seluruh isi variabel hasil ke dalam elemen HTML sehingga dapat diproses sebagai HTML.

3) Hasil



Gambar 5.4 Hasil Eksekusi Latihan 3 – Perulangan dan Function

d. Latihan 4: DOM dan Event

1) Kode Program

Latihan4.html	
1	<!DOCTYPE html>
2	<html>
3	<head>
4	<title>Latihan 4 - DOM dan Event</title>
5	</head>
6	<body>
7	
8	Interaksi JavaScript
9	
10	<button id="tombol">Klik Saya</button>
11	<p id="pesan">Belum ada tindakan.</p>
12	
13	<script>
14	const tombol = document.getElementById("tombol");
15	const pesan = document.getElementById("pesan");
16	
17	tombol.addEventListener("click", function() {
18	pesan.textContent = "Tombol telah diklik!";
19	});
20	</script>
21	
22	</body>
23	</html>

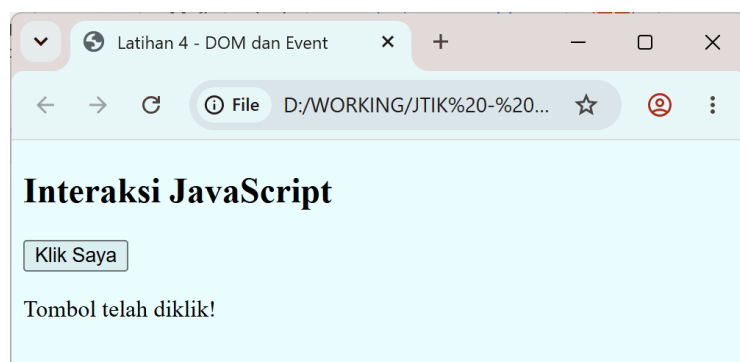
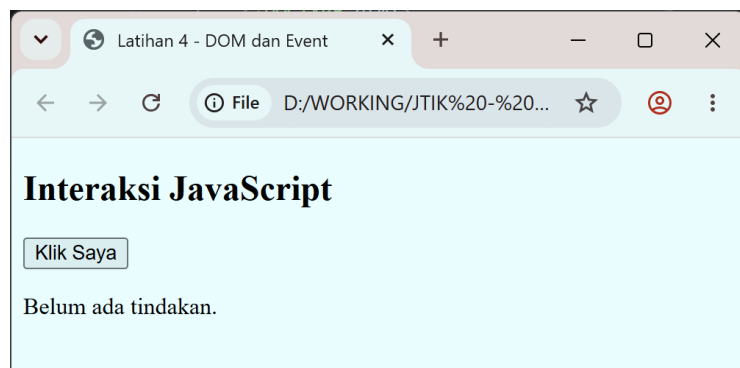
2) Analisis Kode Program

Tabel 5.16 Analisis Program Latihan 4 – DOM dan Event

Kode Program	Analisis
<button id="tombol">Klik Saya</button>	Membuat tombol HTML dengan id="tombol". ID tersebut digunakan JavaScript untuk memilih tombol melalui DOM.
<p id="pesan">Belum ada tindakan.</p>	Membuat elemen paragraf dengan id="pesan" yang digunakan untuk menampilkan informasi kepada pengguna.
const tombol = document.getElemen tById("tombol");	Memilih elemen tombol menggunakan getElementById() dan menyimpannya dalam variabel tombol.

<pre>const pesan = document.getElement ById("pesan");</pre>	Memilih elemen paragraf dan menyimpannya dalam variabel pesan.
<pre>tombol.addEventListener("click", function() {</pre>	Mendaftarkan event click pada tombol. Function di dalamnya akan dijalankan ketika tombol diklik.
<pre>pesan.textContent = "Tombol telah diklik!";</pre>	Mengubah teks pada elemen pesan setelah event click terjadi.
<pre>});</pre>	Menutup function dan proses pendaftaran event.

3) Hasil



Gambar 5.5 Hasil Eksekusi Latihan 4 – DOM dan Event

e. Latihan 5: Form dan Validasi Sederhana

1) Kode Program

Latihan5.html	
1	<!DOCTYPE html>
2	<html>
3	<head>
4	<title>Latihan 5 - Form dan Validasi</title>
5	</head>
6	<body>
7	
8	Form Data Mahasiswa
9	
10	<form id="formMahasiswa">
11	<label>Nama:</label>
12	<input type="text" id="nama">
13	
14	<button type="submit">Kirim</button>
15	</form>
16	
17	<p id="pesan"></p>
18	
19	<script>
20	const form = document.getElementById("formMahasiswa");
21	const nama = document.getElementById("nama");
22	const pesan = document.getElementById("pesan");
23	
24	form.addEventListener("submit", function(event) {
25	event.preventDefault();
26	
27	if (nama.value === "") {
28	pesan.textContent = "Nama harus diisi.";
29	} else {
30	pesan.textContent =
31	"Data berhasil dikirim. Nama: " +
32	nama.value;
33	}
34	});
35	</script>
36	
37	</body>
38	</html>

2) Analisis Kode Program

Tabel 5.17 Analisis Program Latihan 5 – Form dan Validasi

Kode Program	Analisis
<code><form id="formMahasiswa"></code>	Membuat form dengan id="formMahasiswa" yang digunakan sebagai wadah input data mahasiswa.
<code><input type="text" id="nama"></code>	Membuat input teks untuk menerima nama pengguna. id="nama" digunakan JavaScript untuk mengambil nilai input.
<code><button type="submit">Kirim </button></code>	Membuat tombol untuk mengirim form. Karena menggunakan type="submit", tombol memicu event submit pada form.
<code><p id="pesan"></p></code>	Menyediakan elemen untuk menampilkan pesan validasi atau hasil pengolahan data.
<code>const form = document.getElement ById("formMahasiswa ");</code>	Memilih elemen form menggunakan DOM dan menyimpannya dalam variabel form.
<code>const nama = document.getElement ById("nama");</code>	Memilih elemen input nama dan menyimpannya dalam variabel nama.
<code>const pesan = document.getElement ById("pesan");</code>	Memilih elemen paragraf yang digunakan untuk menampilkan pesan.
<code>form.addEventListener ("submit", function(event) {</code>	Mendaftarkan event submit pada form. Function akan dijalankan ketika pengguna menekan tombol Kirim.
<code>event.preventDefault ();</code>	Mencegah perilaku bawaan form sehingga halaman tidak langsung dimuat ulang ketika form dikirim.
<code>nama.value</code>	Mengambil nilai yang dimasukkan pengguna pada input nama.
<code>if (nama.value === "") {</code>	Memeriksa apakah input nama kosong. Operator === membandingkan nilai dan tipe data.
<code>pesan.textContent = "Nama harus diisi.";</code>	Jika input kosong, program menampilkan pesan bahwa nama harus diisi.

<pre>else {</pre>	Menjalankan proses alternatif apabila input nama tidak kosong.
<pre> pesan.textContent = "Data berhasil dikirim..."</pre>	Menampilkan pesan bahwa data berhasil diproses sekaligus menampilkan nama yang dimasukkan pengguna.

3) Hasil



Gambar 5.6 Hasil Eksekusi Latihan 5 – Form dan Validasi

2. STUDI KASUS

a. Studi Kasus 1

Buat sebuah halaman web Sistem Penilaian Mahasiswa yang dapat digunakan untuk memasukkan data dan menentukan hasil penilaian mahasiswa. Halaman harus menyediakan form untuk memasukkan nama mahasiswa, nilai tugas, dan nilai ujian.

JavaScript digunakan untuk mengambil nilai yang dimasukkan pengguna, menghitung nilai rata-rata, kemudian menentukan status kelulusan berdasarkan nilai rata-rata. Jika nilai rata-rata memenuhi batas yang ditentukan, tampilkan keterangan “Lulus”, sedangkan jika tidak memenuhi, tampilkan “Tidak Lulus”.

Program juga harus menampilkan hasil perhitungan secara dinamis pada halaman tanpa membuat halaman baru.

Contoh Hasil Eksekusi Program (Boleh menggunakan tampilan yang berbeda):

The image displays three screenshots of a web application titled "Sistem Penilaian Mahasiswa".

The first screenshot shows the input form with empty fields:

- Nama Mahasiswa:
- Nilai Tugas:
- Nilai Ujian:
- Buttons: and

The second and third screenshots show the results after calculation:

Second Screenshot (Passing):

- Nama Mahasiswa: Ayu Tri Wardani
- Nilai Tugas: 95
- Nilai Ujian: 90
- Buttons: and
- Hasil Penilaian:

 - Nama Mahasiswa: Ayu Tri Wardani
 - Nilai Rata-rata: 92.5
 - Status: Lulus

Third Screenshot (Failing):

- Nama Mahasiswa: Ayu Tri Wardani
- Nilai Tugas: 70
- Nilai Ujian: 50
- Buttons: and
- Hasil Penilaian:

 - Nama Mahasiswa: Ayu Tri Wardani
 - Nilai Rata-rata: 60
 - Status: Tidak Lulus

Gambar 5.7 Contoh Hasil Eksekusi Studi Kasus 1

b. Form Pendaftaran Kegiatan Mahasiswa

Buat sebuah halaman web Pendaftaran Kegiatan Mahasiswa. Halaman tersebut digunakan untuk menerima data mahasiswa yang ingin mengikuti suatu kegiatan.

Form harus menyediakan input nama mahasiswa, NIM, program studi, dan pilihan kegiatan. Setelah tombol Daftar diklik, JavaScript memeriksa kelengkapan data yang dimasukkan. Jika terdapat data yang belum diisi, tampilkan pesan validasi pada halaman. Jika seluruh data telah lengkap, program menampilkan informasi konfirmasi pendaftaran berdasarkan data yang dimasukkan pengguna.

Untuk membuat halaman lebih interaktif, tambahkan sebuah tombol atau elemen yang dapat merespons tindakan pengguna menggunakan event click. Program juga dapat menggunakan function untuk mengatur proses penampilan data.

Contoh Hasil Eksekusi Program (Boleh menggunakan tampilan yang berbeda)

The screenshot shows a web browser window with the title 'Pendaftaran Kegiatan Mahasiswa'. The page has a dark sidebar on the left with the heading 'Kegiatan Mahasiswa' and a list of activity types: 'Seminar', 'Workshop', and 'Lomba'. The main content area is titled 'Form Pendaftaran' and contains the following fields: 'Nama Mahasiswa' (with a placeholder 'Masukkan nama'), 'NIM' (with a placeholder 'Masukkan NIM'), 'Program Studi' (with a placeholder 'Masukkan program studi'), and 'Kegiatan' (a dropdown menu with 'Pilih Kegiatan' as the selected option). At the bottom of the form are two buttons: 'Daftar Sekarang' and 'Pesan'.

This screenshot shows the same web application after the 'Daftar Sekarang' button has been clicked. The form fields remain, but a new section titled 'Data Pendaftaran' has appeared below the form. This section displays the submitted information: 'Nama: Ayo To Rendani', 'NIM: 12345678', 'Program Studi: Pendidikan Teknik Informatika dan Komputer', and 'Kegiatan: Workshop'. Above this data, there is a confirmation message: '✓ Data Pendaftaran' and 'Ayo To Rendani'.

Gambar 5.8 Contoh Hasil Eksekusi Studi Kasus 2

E. PETUNJUK LAPORAN PRAKTIKUM

Laporan praktikum dibuat khusus untuk Tugas Studi Kasus (bukan untuk seluruh latihan), disusun berdasarkan format laporan praktikum yang telah dibagikan.